

Отчёт по лабораторной работе 6

Соловьев Богдан Михайлович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
5	Выводы	17
	Список литературы	18

Список иллюстраций

4.1	График	15
-----	------------------	----

Список таблиц

```
using Pkg
Pkg.activate("C:/Users/bogda/work/study/2026-1/2026-1--study--simul
using Agents, DataFrames, Plots, StatsBase
println("XXXXX XXXXXXXXXXXX: ", Base.active_project())
```

```
XXXXX XXXXXXXXXXXX: C:\Users\bogda\work\study\2026-1\2026-
1--study--simulationmodeling\labs\lab08\project\Project.toml
```

```
Activating project at `C:\Users\bogda\work\study\2026-
1\2026-1--study--simulationmodeling\labs\lab08\project`
```

1 Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR.

Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia.

Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать модель.

2 Задание

Создать рабочий каталог для кода.

Установить необходимые пакеты.

Выполнить предложенный код.

Преобразовать код в литературный стиль.

Сгенерировать из литературного кода:

чистый код;

jupyter notebook;

документацию в формате Quarto.

Выполнить код из jupyter notebook.

Интегрировать документацию в формате Quarto в отчёт.

Добавить в код в литературном стиле вычисление для набора параметров.

Сгенерировать из литературного кода с параметрами:

чистый код;

jupyter notebook;

документацию в формате Quarto.

Выполнить код из jupyter notebook с параметрами.

Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

Модель SIR делит всю популяцию на три взаимосвязанные группы (компарменты), что отражено в её названии:

S — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться.

I — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию.

R — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи.

Основная цель модели: не предсказать судьбу конкретного человека, а понять общую динамику эпидемии — будет ли она разрастаться,

как быстро, сколько людей в итоге переболеет, как влияют карантинные меры.

4 Выполнение лабораторной работы

Программа реализует стохастическую SIR-модель на основе дискретных агентов (индивидов)

с использованием библиотеки ConcurrentSim для событийно-ориентированного моделирования.

Вспомогательные функции мутации массивов статистики

`increment!(a)` — добавляет в конец массива значение на 1 больше последнего.

`decrement!(a)` — добавляет значение на 1 меньше последнего.

`carryover!(a)` — дублирует последнее значение.

Используются для фиксации изменений численности восприимчивых (S_a), инфицированных (I_a) и выздоровевших (R_a) в момент событий.

Структуры данных

SIRPerson — агент с полями `id` (уникальный идентификатор) и `status` (состояние :S, :I, :R).

SIRModel — контейнер состояния модели: объект симуляции, параметры (β , γ , δ), временные ряды (t_a , S_a , I_a , R_a) и список всех агентов.

Функции обновления статистики

`infection_update!()` — фиксирует заражение: добавляет текущее время, уменьшает S, увеличивает I, R без изменений.

`recovery_update!()` — фиксирует выздоровление: добавляет время, S без изменений, уменьшает I, увеличивает R.

Основной процесс агента (`live`)

Функция с макросом `@resumable` описывает жизненный цикл индивида:

1. **Восприимчивый (:S)** — ждёт случайное время (экспоненциальное с параметром $1/c$),

случайно выбирает другого индивида; если тот инфицирован — с вероятностью β заражается.

2. **Инфицированный (:I)** — ждёт случайное время выздоровления (экспоненциальное с параметром $1/\gamma$) и переходит в состояние :R.

Функции управления моделью

MakeSIRModel(u0, p) — создаёт модель с начальными условиями $[S_0, I_0, R_0]$ и параметрами $[\beta, c, \gamma]$.

activate(m) — регистрирует процессы всех агентов в симуляции.

sir_run(m, tf) — запускает симуляцию до момента времени tf .

out(m) — возвращает результаты в виде DataFrame с колонками t, S, I, R .

```
using DrWatson
@quickactivate "project"
include(sourcedir("sir_model.jl"))
using Random, StatsPlots, BenchmarkTools
```

```
└ Warning: DrWatson could not find find a project file by recursively cl
└ (given dir: C:\Users\bogda)
└ @ DrWatson C:\Users\bogda\.julia\packages\DrWatson\2QF5p\src\projec
```

Параметры модели

```
tmax = 40.0
u0 = [990, 10, 0] # S, I, R
p = [0.05, 10.0, 0.25] #  $\beta, c, \gamma$ 

Random.seed!(1234)
```

TaskLocalRNG()

Запуск модели

```
des_model = MakeSIRModel(u0, p)
activate(des_model)
sir_run(des_model, tmax)
data_des = out(des_model)
```

	t	S	I	R	
	Float64	Int64	Int64	Int64	
1	0.0	990	10	0	
2	0.110851	989	11	0	
3	0.249585	988	12	0	
4	0.314392	987	13	0	
5	0.418738	987	12	1	
6	0.457041	987	11	2	
7	0.528895	986	12	2	
8	0.619502	985	13	2	
9	0.657704	984	14	2	
10	0.807485	983	15	2	
11	0.913646	983	14	3	
12	0.976249	982	15	3	
13	1.07816	982	14	4	
14	1.09161	982	13	5	
15	1.1487	982	12	6	
16	1.17259	981	13	6	
17	1.2789	980	14	6	
18	1.30828	980	13	7	
19	1.37686	979	14	7	
20	1.44574	978	15	7	
21	1.45046	977	16	7	
22	1.51802	977	15	8	
23	1.63961	976	16	8	
24	1.71622	976	15	9	
25	2.05024	976	14	10	
26	2.0729	975	15	10	
27	2.20607	974	16	10	
28	2.30437	974	15	11	
29	2.37251	973	16	11	11
30	2.56655	973	15	12	
...	...	...	...	...	

Визуализация

```
@df data_des plot(  
    :t,  
    [:S :I :R],  
    labels = ["S" "I" "R"],  
    xlab = "time",  
    ylab = "population",  
    title = "SIR model",  
)  
savefig(plotsdir("sir_des.png"))
```

```
"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--  
simulationmodeling\\labs\\lab08\\project\\plots\\sir_des.png"
```

В скрипте запуска есть следующие входные данные:

$t_{\max} = 40.0$ — длительность симуляции.

$u_0 = [990, 10, 0]$ — начальные количества S, I, R.

$p = [0.05, 10.0, 0.25]$ — β , c , γ .

```
using DrWatson  
@quickactivate "project"  
include(sourcedir("sir_model.jl"))  
using Random, StatsPlots, BenchmarkTools
```

```
└ Warning: DrWatson could not find find a project file by recursively cl  
  (given dir: C:\Users\bogda)  
└ @ DrWatson C:\Users\bogda\.julia\packages\DrWatson\2QF5p\src\proje
```

Параметры модели

```
tmax = 40.0
u0 = [990, 10, 0]      # S, I, R
p = [0.05, 10.0, 0.25] #  $\beta$ , c,  $\gamma$ 

Random.seed!(1234)
```

TaskLocalRNG()

Запуск модели

```
des_model = MakeSIRModel(u0, p)
activate(des_model)
sir_run(des_model, tmax)
data_des = out(des_model)
```

	t	S	I	R	
	Float64	Int64	Int64	Int64	
1	0.0	990	10	0	
2	0.110851	989	11	0	
3	0.249585	988	12	0	
4	0.314392	987	13	0	
5	0.418738	987	12	1	
6	0.457041	987	11	2	
7	0.528895	986	12	2	
8	0.619502	985	13	2	
9	0.657704	984	14	2	
10	0.807485	983	15	2	
11	0.913646	983	14	3	
12	0.976249	982	15	3	
13	1.07816	982	14	4	
14	1.09161	982	13	5	
15	1.1487	982	12	6	
16	1.17259	981	13	6	
17	1.2789	980	14	6	
18	1.30828	980	13	7	
19	1.37686	979	14	7	
20	1.44574	978	15	7	
21	1.45046	977	16	7	
22	1.51802	977	15	8	
23	1.63961	976	16	8	
24	1.71622	976	15	9	
25	2.05024	976	14	10	
26	2.0729	975	15	10	
27	2.20607	974	16	10	
28	2.30437	974	15	11	
29	2.37251	973	16	11	14
30	2.56655	973	15	12	
...	...	...	...	...	

Визуализация

```
@df data_des plot(  
    :t,  
    [:S :I :R],  
    labels = ["S" "I" "R"],  
    xlab = "Время",  
    ylab = "Численность",  
    title = "Дискретно-событийная SIR модель",  
)  
savefig(plotsdir("sir_des.png"))
```

"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--simulationmodeling\\labs\\lab08\\project\\plots\\sir_des.png"

Здесь показан получившийся график (рис. 4.1)

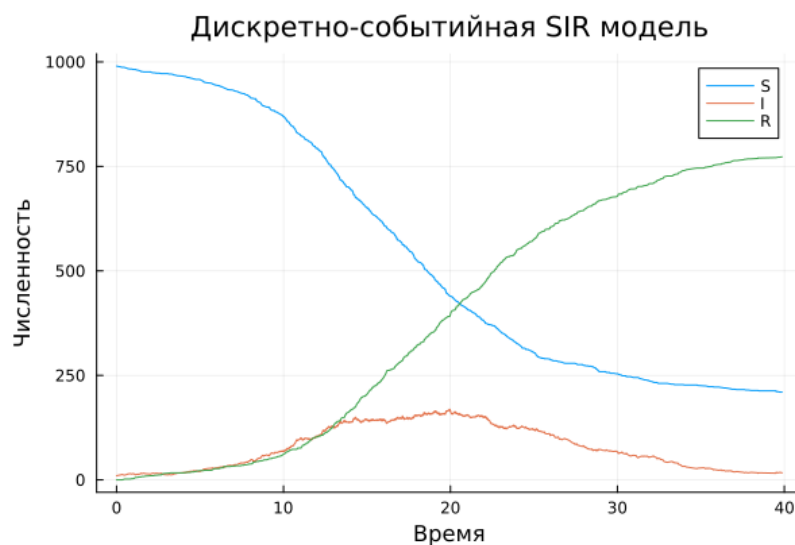


Рисунок 4.1: График

Программа моделирует стохастическое распространение инфекции в полностью перемешанной популяции конечного размера.

В отличие от детерминированной модели (системы ОДУ), здесь наблюдаются случайные флуктуации и возможны ранние вымирания инфекции при малом начальном числе заражённых.

1. Инициализация — создаются объекты-агенты (`SIRPerson`) с начальными статусами (`:S`, `:I`, `:R`), инициализируются массивы статистики (`ta`, `Sa`, `Ia`, `Ra`).

2. Планирование процессов — для каждого агента создаётся параллельный процесс (`@process`), реализующий его жизненный цикл (функция `live`).

3. Запуск симуляции — `ConcurrentSim.run` продвигает виртуальное время, обрабатывая события в хронологическом порядке. Каждый `@yield timeout` добавляет событие в календарь; при наступлении события выполняется соответствующий код в `live` (возможно, порождая новые события).

4. Сбор статистики — в момент изменения статуса агента обновляются временные ряды (`Sa`, `Ia`, `Ra`) и фиксируется текущее время в `ta`.

5. Завершение — по достижении конечного времени `tf` симуляция останавливается; накопленные ряды экспортируются в `DataFrame`.

6. Постобработка — построение графиков динамики эпидемии, сохранение результатов, анализ вымираний.

5 Выводы

Я изучил дискретно-событийный подход у имитационному моделированию на примере классической модели SIR

Список литературы